

Procédure d'installation de Passerelle sur un serveur

Application	Passerelle (nom technique onboardcc) — plateforme de suivi des candidatures de volontaires
Document	Procédure d'installation (déploiement initial ou réinstallation complète)
Version	2.0 — 25/09/2026
Public	Exploitant informatique chargé d'installer le serveur (niveau technique non expert accepté : chaque geste est expliqué)
Document complémentaire	2-procedure_maintenance_passerelle.md — gestes de maintenance une fois l'application en service

 **Valeurs à remplacer.** Les noms entre chevrons **< >** sont propres à chaque installation : à remplacer par les vraies valeurs **avant** de copier une commande.

Valeur	Signification
<COMPTE>	Identifiant du compte d'hébergement (préfixe de l'utilisateur, de la base, des dossiers)
<HOTE_SSH>	Nom d'hôte du serveur pour la connexion SSH
<SERVEUR>	Nom interne du serveur (affiché dans l'invite du terminal — informatif)
<DOMAINE>	Adresse de l'application, sans https:// (ex. passerelle.exemple.org)
<DOSSIER_PUBLIC>	Dossier public du domaine, servi par Apache (ex. /home/<COMPTE>/<DOMAINE>)
<CHEMIN_CLE_SSH>	Emplacement de la clé privée SSH sur le poste d'administration

Introduction

Objectif de cette procédure

Cette procédure permet d'**installer l'application Passerelle sur un serveur de production, en partant de rien**, ou de la **réinstaller** sur un serveur déjà utilisé. À la fin, l'application est accessible en HTTPS à l'adresse **https://<DOMAINE>**, avec sa base de données, ses envois d'emails et un premier compte administrateur.

Elle décrit un hébergement mutualisé de type **cPanel** avec **Node.js Selector** et **PostgreSQL**. Les mêmes principes valent pour tout serveur Node.js + PostgreSQL ; seules les manipulations dans cPanel sont à transposer.

Les grandes étapes

#	Étape	Ce qu'on y fait	Résultat
1	Accéder au serveur	Configurer l'accès SSH (clé + autorisation d'adresse IP) et s'y connecter	Un terminal ouvert sur le serveur
2	Application Node.js et code source	Créer l'application Node.js, récupérer le code depuis GitHub, installer ses dépendances, compiler le backend	Le code est sur le serveur et compilé
3	Base de données PostgreSQL	Créer la base, y charger le schéma et les référentiels	Une base prête, sans donnée personnelle
4	Variables d'environnement	Renseigner les réglages (connexion à la base, secret, barème de score...)	L'application sait comment fonctionner
5	Stockage des pièces jointes	Créer le dossier <code>uploads/</code>	Les pièces jointes des candidats ont un emplacement
6	Envoi d'emails (SMTP)	Créer un compte email d'envoi et le déclarer	Invitations et mots de passe oubliés partent
7	Démarrage et publication	Démarrer le backend, compiler et publier l'interface	<code>https://<DOMAINE></code> affiche l'application
8	Premier compte administrateur	Créer le compte qui permettra de créer tous les autres	On peut se connecter
9	Sécurisation	Contrôler la liste de sécurité, le certificat HTTPS	Un serveur prêt à recevoir de vrais utilisateurs
10	Recette	Vérifier les fonctions essentielles, nettoyer les données de test	Mise à disposition

Durée indicative d'une installation complète pour un exploitant qui suit la procédure : **environ une demi-journée** (*estimation*).

Première installation ou réinstallation ?

Situation	Étapes à faire
Première installation	Toutes, dans l'ordre
Réinstallation sur un serveur déjà installé (nouvelle version, base à repartir de zéro)	Étape 2 (cas B), étape 3 (avec le nettoyage de la base), étape 4 (comparer les variables), étape 7 (sans les règles Apache), étape 8. Les étapes 1, 5, 6 et 9 sont déjà faites : à vérifier seulement

Comment lire cette procédure

Chaque étape suit le même modèle : « **Avant de commencer** » (ce qui doit être fait ou disponible), puis des instructions **numérotées**. Une instruction précise **où** la taper.

Où on tape	Comment le reconnaître	Ce qu'on y tape
PowerShell / terminal du poste	Invite du poste de l'exploitant (ex. PS C:\Users\...>)	Uniquement la commande ssh de connexion
bash (terminal du serveur)	Invite qui se termine par \$, ex. [<COMPTE>@<SERVEUR> ~]\$	cd, git, npm, node, psql...
psql (base de données)	Invite qui se termine par => , ex. <COMPTE>_passerelle=>	Uniquement des requêtes SQL, terminées par ; , et les commandes commençant par \
cPanel / navigateur	Fenêtre du navigateur	Clics dans l'interface de l'hébergeur

-  **Encadré « ampoule »** : explication complémentaire sur le geste qui précède.
-  **Encadré « attention »** : risque ou erreur fréquente à éviter.
- Chaque bloc de code correspond à **une commande ou une série de commandes à taper ensemble**.
- Une requête SQL tapée dans bash, ou un **cd** tapé dans psql, provoque une erreur : bien vérifier l'invite.

 **Règle pour tout mot de passe créé pour ce projet** (base de données, compte email, administrateur) : **uniquement des lettres, des chiffres et les caractères _ - . —** jamais d'autres caractères spéciaux (**# ? & @ : / % ! \$ '...**). Ces caractères sont interprétés par le terminal ou par les adresses de connexion (**#, ?, &** cassent la lecture d'une adresse de base de données, **!** est un rappel d'historique dans bash) et provoquent des échecs très difficiles à relier à leur vraie cause. La longueur compte bien plus que la variété des symboles : **12 caractères minimum** recommandés.

À préparer avant de commencer

- ☐ Accès à l'interface **cPanel** de l'hébergement, avec **Node.js Selector** et **PostgreSQL** disponibles.
- ☐ Un **domaine ou sous-domaine** créé et pointant vers l'hébergement (**<DOMAINE>**).
- ☐ Un **poste d'administration** avec un client **SSH** (inclus dans Windows 10/11, macOS et Linux).
- ☐ L'accès **SSH** activé sur l'offre d'hébergement.
- ☐ Un **mot de passe** pour l'utilisateur de la base, un autre pour le compte email d'envoi, un autre pour le premier administrateur (règle ci-dessus).
- ☐ L'**identité du premier administrateur** (nom, prénom, adresse email).

Étape 1 — Accéder au serveur

Ce que fait cette étape : permettre à l'exploitant de piloter le serveur à distance, de façon sécurisée, depuis son poste. Toutes les étapes suivantes reposent sur cet accès.

Avant de commencer :

- ☐ Accès à cPanel.
- ☐ Poste d'administration avec un client SSH.

 **SSH** (*Secure Shell*) permet d'exécuter des commandes sur le serveur depuis son poste, à travers une connexion chiffrée. Contrairement au FTP, qui ne fait que transférer des fichiers, SSH permet de configurer Node.js, d'interroger la base de données ou de lire les journaux.

1.1 Générer une paire de clés

(où : *cPanel*, dans le navigateur)

1. Ouvrir cPanel → **Sécurité** → **Accès SSH** → **Gérer les clés SSH** → **Générer une nouvelle clé**.
2. Renseigner : un nom (ex. `passerelle_admin`), type **RSA**, taille **4096 bits**, une **phrase secrète** (obligatoire, à conserver précieusement : elle est demandée à chaque connexion).
3. Valider la génération.

💡 **Une paire de clés, comment ça marche ?** La **clé privée** reste uniquement sur le poste qui se connecte, elle n'est jamais partagée. La **clé publique** est déposée sur le serveur. À la connexion, le serveur vérifie que la clé privée présentée correspond à la clé publique enregistrée, sans qu'aucun mot de passe ne circule. Analogie : la clé privée est la clé de la maison (toujours sur soi), la clé publique est la serrure côté serveur.

1.2 Récupérer la clé privée

(où : *cPanel*)

1. Retourner sur la page **Gérer les clés SSH**.
2. Dans la section **Clés privées**, repérer la clé créée et cliquer sur **Afficher/Télécharger**, puis enregistrer le fichier sur le poste, dans un dossier réservé (sous Windows : `C:\Users\<UTILISATEUR>\.ssh\`).

⚠ Le journal affiché juste après la génération (empreinte, dessin « randomart ») **n'est pas la clé** : la clé privée se télécharge depuis la section **Clés privées**.

1.3 Autoriser la clé publique

(où : *cPanel*)

1. Dans **Gérer les clés SSH**, section **Clés publiques**, repérer la clé et cliquer sur **Gérer**.
2. Cliquer sur **Authorize** (Autoriser).

💡 **Générer n'est pas autoriser.** *Générer* crée la paire de clés ; *autoriser* dit au serveur « je fais confiance à cette clé publique ». Oublier l'autorisation est une erreur courante : la clé semble prête mais la connexion échoue.

1.4 Autoriser l'adresse IP du poste

(où : *cPanel*)

1. Dans la barre de recherche de cPanel, taper **Autorisation SSH** (outil de l'hébergeur).
2. Cliquer sur **Ajouter une autorisation**, laisser l'adresse IP publique détectée, choisir le port **22 - SSH**, puis **Confirmer**.

💡 **Deux protections indépendantes.** La clé SSH prouve **qui** se connecte ; la liste d'adresses autorisées contrôle **d'où** on a le droit de se connecter. Les deux doivent être en place. Sans cette autorisation, la connexion échoue par un délai d'attente dépassé (`Connection timed out`).

⚠ Si la connexion se met à expirer plus tard, l'adresse IP publique du poste a probablement changé (adresse dynamique) : revenir sur **Autorisation SSH** et ajouter la nouvelle adresse.

1.5 Se connecter au serveur

(où : *PowerShell* ou *terminal du poste*)

1. Ouvrir un terminal sur le poste d'administration, puis taper (en adaptant le chemin de la clé) :

```
ssh -i <CHEMIN_CLE_SSH> <COMPTE>@<HOTE_SSH>
```

2. Saisir la **phrase secrète** de la clé (rien ne s'affiche pendant la saisie : c'est normal).

Résultat attendu : l'invite change et affiche `[<COMPTE>@<SERVEUR> ~]$`. **Vous êtes maintenant dans bash, sur le serveur** : toutes les commandes « bash » des étapes suivantes se tapent dans cette fenêtre.

💡 Cette fenêtre reste ouverte pendant toute l'installation. Si elle est fermée, il suffit de refaire la commande `ssh` ci-dessus.

Étape 2 — Application Node.js et code source

Ce que fait cette étape : créer l'application Node.js chez l'hébergeur, récupérer le code de Passerelle depuis GitHub, installer les composants dont il dépend et compiler le backend (la partie serveur).

Avant de commencer :

- ☐ Étape 1 terminée : une session SSH est ouverte (invite `[...]$`).
- ☐ Le dépôt GitHub <https://github.com/DamienMoreauConsultant/onboardcc> est **à jour** : ouvrir sa page dans un navigateur et vérifier que le dernier commit (message et date) correspond à la version à installer.

⚠ **Le serveur ne peut récupérer que ce qui a été poussé sur GitHub.** Si le développeur a des modifications non poussées, les lui faire pousser d'abord.

💡 **Architecture retenue.** Le projet contient deux applications : le **backend** (serveur Node.js, sous `/api`) et le **frontend** (l'interface, des fichiers statiques). Les deux partagent le même domaine : `https://<DOMAINE>/` affiche l'interface, `https://<DOMAINE>/api/...` est traité par Node.js. Seul le backend est géré par Node.js Selector ; l'interface est servie directement par Apache (étape 7).

2.1 Créer l'application Node.js

(où : *cPanel*)

1. Ouvrir cPanel → **Setup Node.js App** → **Créer une application**.
2. Renseigner les champs suivants, puis **Créer** :

Champ	Valeur	Pourquoi
Version de Node.js	24 (la plus récente proposée)	Version utilisée pour développer et tester Passerelle
Mode d'application	Production	Conditions réelles de fonctionnement

Champ	Valeur	Pourquoi
Racine de l'application	<code>passerelle-api</code>	Dossier qui contiendra le code
URL de l'application	<code><DOMAINE></code> avec le sous-chemin <code>api</code>	Isole le backend sous <code>/api</code> et laisse <code>/</code> à l'interface
Fichier de démarrage	<code>artifacts/api-server/dist/index.mjs</code>	Point d'entrée compilé du backend

⚠ Ne pas cliquer sur le bouton « **Exécuter NPM Install** » proposé par cPanel : le projet impose l'outil `pnpm` (voir 2.3) et bloque volontairement `npm`.

- Repérer, en haut de la page de l'application, la ligne « **Enter to the virtual environment** » : elle contient la commande `source .../activate` utilisée plus bas.

2.2 Récupérer le code

(où : `bash`, sur le serveur)

1 — Regarder si le dossier contient déjà du code

```
ls -A ~/passerelle-api
```

- Aucune ligne affichée** (ou « No such file or directory ») → **cas A**, première installation.
- Des fichiers sont listés** → **cas B**, le code est déjà présent.

2 — Cas A : première installation — cloner le dépôt

```
git clone https://github.com/DamienMoreauConsultant/onboardcc.git ~/passerelle-api
```

💡 Le dépôt est public : **aucune authentification** n'est demandée. Le dossier `~/passerelle-api` n'a pas besoin d'être créé avant : `git clone` le crée lui-même. S'il existe déjà mais est **vide** (créé par cPanel à l'étape 2.1), cela fonctionne aussi.

2 — Cas B : le code est déjà présent — récupérer les nouveautés

```
source /home/<COMPTE>/nodeenv/passerelle-api/24/bin/activate && cd ~/passerelle-api
git status
git pull
```

- `git status` doit indiquer qu'il n'y a **aucune modification à valider** (`nothing to commit`, ou seulement des fichiers « untracked ») : un serveur ne contient jamais de modification locale.
- `git pull` remplace le code par la dernière version de GitHub, **sans toucher** au dossier `uploads/` (les pièces jointes) ni aux fichiers que Git ne suit pas.

⚠ **Ne pas recloner dans le cas B.** `git clone` ne remplace rien : sur un dossier **non vide** il refuse (`destination path ... already exists and is not an empty directory`). Et supprimer le dossier pour recloner détruirait `uploads/`. Pour repartir malgré tout d'un dossier neuf : le mettre de côté (`mv ~/passerelle-api ~/passerelle-api_ancien`), cloner (cas A), recopier `uploads/`, puis ne supprimer l'ancien dossier qu'après vérification de la nouvelle installation.

💡 **`git status` doit rester propre.** Ne jamais faire `git add` ni `git commit` sur le serveur : il ne fait que **récupérer** (`pull`), jamais **envoyer** (`push`). Cette règle à sens unique garantit qu'aucune synchronisation ne peut rien écraser par erreur.

2.3 Installer les dépendances

(où : `bash`, sur le serveur)

💡 **Qu'est-ce qu'une dépendance ? `npm` ? `pnpm` ?** Passerelle s'appuie sur des centaines de briques logicielles déjà écrites par d'autres (formulaires, connexion à PostgreSQL, envoi d'emails...), appelées **dépendances** et listées dans `package.json`. **`pnpm`** est l'outil qui les télécharge et les range dans un dossier `node_modules`. C'est l'équivalent de `npm`, plus rapide et plus économe en espace disque, et c'est celui que ce projet impose.

1 — Activer l'environnement Node du projet

```
source /home/<COMPTE>/nodeenv/passerelle-api/24/bin/activate && cd ~/passerelle-api
```

Résultat attendu : `(24)` apparaît au début de l'invite. **À refaire à chaque nouvelle connexion SSH**, sinon `pnpm: command not found`.

2 — Première installation : installer `pnpm`, puis les dépendances

```
npm install -g pnpm
pnpm install
pnpm approve-builds
pnpm install
```

💡 `pnpm approve-builds` autorise un script d'installation légitime (`esbuild`) que `pnpm` bloque par défaut par sécurité ; le deuxième `pnpm install` l'exécute. Dans la version livrée, cette autorisation est déjà enregistrée : la commande peut n'afficher aucune demande.

⚠ Après ces commandes, contrôler `git status`. Si le fichier `pnpm-workspace.yaml` apparaît modifié, ne rien valider : l'annuler avec `git restore pnpm-workspace.yaml`. Un serveur ne doit jamais avoir de modification locale (voir 2.2), sinon le prochain `git pull` échoue.

2 — Réinstallation (cas B) : `pnpm install` n'est nécessaire que si `git pull` a modifié `package.json`, `pnpm-lock.yaml` ou `pnpm-workspace.yaml`. Dans le doute, le lancer : il répond `Already up to date` si rien n'a changé.

```
pnpm install
```

2.4 Compiler le backend

(où : *bash*, sur le serveur)

```
cd ~/passerelle-api/artifacts/api-server
pnpm run build
```

Résultat attendu : une liste de fichiers `dist/...` puis ⚡ **Done**. Le fichier `dist/index.mjs` est créé. Un avertissement sur la taille de ce fichier est normal.

💡 Le code source (`src/`) ne suffit pas : le serveur exécute le fichier compilé `dist/index.mjs`. Il doit donc être recompilé après chaque changement du code.

⚠ L'application ne démarre pas encore : il manque la base de données (étape 3) et les variables d'environnement (étape 4). Ne pas la redémarrer avant.

Étape 3 — Base de données PostgreSQL

Ce que fait cette étape : créer la base et y charger, avec **un seul fichier**, le schéma complet et les référentiels validés par la DCC (pays, compétences, langues...). Ce fichier ne contient **aucune donnée personnelle**.

Avant de commencer :

- ☐ Étape 2 terminée : le code est sur le serveur (dossier `~/passerelle-api`, dont `documentation/`).
- ☐ Un mot de passe choisi pour l'utilisateur de la base (règle des mots de passe, page 1).

3.1 Créer la base et l'utilisateur (première installation)

(où : *cPanel*)

Réinstallation : la base et l'utilisateur existent déjà, passer directement au **3.2**.

- Ouvrir cPanel → **Assistant de base de données PostgreSQL**.
- Créer la base** (ex. `passerelle`), puis **l'utilisateur** (ex. `passerelle_app`) avec son mot de passe, puis lui **attribuer tous les privilèges** sur la base.
- Noter les trois noms obtenus.

💡 Sur un hébergement mutualisé, les noms sont automatiquement préfixés par l'identifiant du compte : `passerelle` devient `<COMPTE>_passerelle` et `passerelle_app` devient `<COMPTE>_passerelle_app`. C'est avec ces noms complets que l'on se connecte.

💡 L'Assistant enchaîne création de la base, de l'utilisateur et attribution des droits : c'est plus sûr que l'interface manuelle, où il faut penser aux trois opérations séparément.

3.2 Ouvrir la base

(où : *bash*, sur le serveur)

```
cd ~/passerelle-api
psql -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle
```

Saisir le mot de passe de l'utilisateur de la base (rien ne s'affiche). Résultat attendu : **psql (9.6.22)** (ou la version du serveur) puis l'invite **<COMPTE>_passerelle=>**. **Vous êtes maintenant dans psql** : les instructions 3.3 à 3.6 s'y tapent, jusqu'à la sortie.

3.3 Noter la version de PostgreSQL

(où : *psql*)

```
SELECT version();
```

Noter le numéro de version affiché (ex. **PostgreSQL 9.6.22** ou **PostgreSQL 16.4**).

💡 La version détermine le fichier à charger en 3.5 : le code est développé avec PostgreSQL 16, et les serveurs plus anciens (avant la version 10) ne comprennent pas certaines instructions. Ne jamais supposer qu'un serveur est récent.

3.4 Vider la base (réinstallation uniquement)

(où : *psql*)

Première installation : passer au **3.5**. Sur une base déjà utilisée, ce bloc **supprime toutes les tables** (et donc toutes les données, comptes compris) pour repartir d'une base vide. Faire une **sauvegarde** avant (voir la procédure de maintenance, §1).

```
DO $$
DECLARE r RECORD;
BEGIN
  FOR r IN (SELECT tablename FROM pg_tables WHERE schemaname = 'public') LOOP
    EXECUTE 'DROP TABLE IF EXISTS public.' || quote_ident(r.tablename) || '
CASCADE';
  END LOOP;
  FOR r IN (SELECT sequence_name FROM information_schema.sequences WHERE
sequence_schema = 'public') LOOP
    EXECUTE 'DROP SEQUENCE IF EXISTS public.' || quote_ident(r.sequence_name) || '
CASCADE';
  END LOOP;
END $$;
```

Résultat attendu : **DO**, précédé de messages **NOTICE** sur des liens supprimés en cascade (normal).

💡 Sur un hébergement mutualisé, l'utilisateur de la base n'est pas propriétaire du schéma **public** : **DROP SCHEMA public** échoue. On supprime donc les tables et les séquences une à une, ce que fait ce

bloc.

3.5 Charger le fichier d'installation

(où : *psql*)

Le fichier existe en **deux versions** dans le dossier `documentation/` du code (donc déjà sur le serveur) ; choisir **une seule** selon la version notée en 3.3 :

Version de PostgreSQL du serveur	Fichier à charger
10 ou plus récente (10, 11, ... 16)	<code>prompt_0_sql_versionAvantProd_20260918.sql</code>
9.6 ou plus ancienne	<code>prompt_0_sql_versionAvantProd_20260918_pg96.sql</code>

Taper **une seule** de ces deux commandes :

```
\i documentation/prompt_0_sql_versionAvantProd_20260918.sql
```

```
\i documentation/prompt_0_sql_versionAvantProd_20260918_pg96.sql
```

Le chargement prend quelques secondes et affiche de nombreuses lignes. Résultat attendu : il se termine par **COMMIT**, sans aucune ligne contenant **ERREUR** ou **ERROR**.

💡 La version `_pg96` est identique à l'autre, sauf 4 éléments retirés parce que les anciens serveurs (avant la version 10) ne les comprennent pas. Charger la version d'origine sur un serveur ancien provoque des erreurs (`syntax error, role "postgres" does not exist`).

⚠️ **Ne charger le fichier qu'une seule fois.** Relancé sur une base déjà chargée, il affiche des centaines d'erreurs « la relation ... existe déjà » (sans dégât, mais la base n'est plus garantie saine). En cas de doute ou d'erreur : refaire le nettoyage (3.4) puis **un seul** chargement. En cas d'erreur en cours de chargement, **ne pas poursuivre** avec une base à moitié chargée.

⚠️ **Poste Windows uniquement** — si `psql` était lancé depuis un poste Windows (et non depuis la session SSH du serveur), forcer d'abord l'encodage : `$env:PGCLIENTENCODING = "UTF8"` dans PowerShell avant de lancer `psql`. Depuis la session SSH du serveur, ce n'est pas nécessaire.

3.6 Vérifier le chargement, puis quitter psql

(où : *psql*)

⚠️ **Juste après le chargement, la session `psql` n'a plus de « chemin de recherche »** : une requête `SELECT ... FROM pays` répond « la relation pays n'existe pas » alors que la table est bien là. Toujours préfixer les tables par **public**. dans les requêtes ci-dessous.

```
SELECT count(*) FROM public.pays;           -- attendu : 42
SELECT count(*) FROM public.competences;    -- attendu : 54
```

```
SELECT count(*) FROM public.etat_candidat;      -- attendu : 11
SELECT count(*) FROM public.aide_contextuelle;  -- attendu : 36
```

Quitter psql pour revenir dans bash :

```
\q
```

Résultat attendu : l'invite redevient [`<COMPTE>@<SERVEUR> ...`]\$. Il manque encore le premier compte administrateur (aucune donnée de compte dans le fichier) : voir l'**étape 8**.

Étape 4 — Variables d'environnement

Ce que fait cette étape : donner à l'application ses réglages : accès à la base, secret de sécurité, adresses, barème du score de correspondance.

Avant de commencer :

- ☐ Étape 3 terminée (base créée et chargée).
- ☐ Le nom complet de l'utilisateur et de la base (étape 3.1) et son mot de passe.
- ☐ Une session SSH ouverte avec l'environnement Node activé (étape 2.3).

 **Pourquoi pas un fichier .env sur le serveur ?** Sur cet hébergement, les réglages se saisissent dans cPanel : ils y sont protégés et l'application redémarre automatiquement à l'enregistrement. Aucun fichier contenant des secrets ne doit exister dans un dossier accessible depuis le web.

4.1 Ouvrir le panneau des variables

(où : cPanel)

Ouvrir cPanel → **Setup Node.js App** → cliquer sur l'application → section **Environment variables**.

4.2 Générer le secret de sécurité (JWT_SECRET)

(où : bash, sur le serveur)

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Copier la longue suite de caractères affichée : elle sert de valeur à `JWT_SECRET` (4.3).

 Ce secret signe les jetons de connexion des utilisateurs. Il doit être **propre à cette installation** : ne jamais réutiliser celui d'un autre environnement, ne jamais le communiquer.

4.3 Ajouter les variables propres à l'installation

(où : cPanel)

Pour chaque ligne du tableau : **Ajouter une variable**, saisir le nom exact et la valeur.

Nom	Valeur	Remarque
DATABASE_URL	postgresql://<COMPTE>_passerelle_app:<MOT_DE_PASSE>@localhost:5432/<COMPTE>_passerelle	Mot de passe de la base (règle des mots de passe : aucun caractère spécial)
JWT_SECRET	(la valeur générée en 4.2)	Secret propre à l'installation
JWT_EXPIRE_HOURS	8	Durée d'une session de connexion
FRONTEND_URL	https://<DOMAINE>	Adresse publique de l'application
UPLOAD_DIR	/home/<COMPTE>/passerelle-api/uploads	Dossier des pièces jointes (créé à l'étape 5)
PIECE_JOINTE_STORAGE_URL	https://<DOMAINE>/uploads/	Adresse publique des pièces jointes
PORT	3000	Indispensable sur cet hébergement (voir l'encadré)

⚠️ PORT est indispensable : sans elle, l'application plante au démarrage (**PORT environment variable is required**) et cPanel n'affiche qu'un message générique « Could not spawn process... exited prematurely ».

4.4 Ajouter le barème du score (MATCH_SCORE_*)

(où : *bash*, puis *cPanel*)

1. Afficher la liste de référence des variables, dans bash :

```
cat ~/passerelle-api/artifacts/api-server/.env.example
```

2. Dans cPanel, ajouter **toutes les variables commençant par MATCH_SCORE_** avec les valeurs indiquées.

⚠ **Aucune variable ne doit manquer.** Le calcul du score vérifie toutes les variables `MATCH_SCORE_*` d'un coup : si une seule manque, **tous** les calculs échouent avec l'erreur `Configuration de scoring absente ou invalide : NOM_DE_LA_VARIABLE`. En réinstallation, comparer soigneusement la liste de `cPanel` avec `.env.example` : ajouter les variables nouvelles et supprimer celles qui n'y figurent plus.

💡 Le fichier `.env.example` fait référence : il est tenu à jour avec le code. Les variables `SMTP_*` sont ajoutées à l'étape 6.

4.5 Enregistrer

(où : `cPanel`)

Cliquer sur **Enregistrer**. L'application redémarre automatiquement.

Étape 5 — Stockage des pièces jointes

Ce que fait cette étape : créer le dossier qui recevra les pièces jointes déposées par les utilisateurs (PDF, images), et empêcher qu'on puisse en lister le contenu depuis un navigateur.

Avant de commencer :

- ☐ Étape 4 terminée, en particulier `UPLOAD_DIR=/home/<COMPTE>/passerelle-api/uploads`.
- ☐ Une session SSH ouverte (invite `bash`).

Réinstallation : le dossier existe déjà, avec les pièces jointes des candidats. **Ne pas le supprimer**. Vérifier seulement sa présence : `ls -la ~/passerelle-api/uploads/`.

1 — Créer le dossier et le protéger

(où : `bash`, sur le serveur)

```
mkdir -p /home/<COMPTE>/passerelle-api/uploads
chmod 755 /home/<COMPTE>/passerelle-api/uploads
echo "Options -Indexes" > /home/<COMPTE>/passerelle-api/uploads/.htaccess
```

💡 `Options -Indexes` empêche de **lister** le contenu du dossier en y naviguant : chaque pièce jointe reste accessible par son lien précis, jamais « en vrac ». Prendre ce réflexe pour tout dossier de fichiers.

💡 L'écriture des fichiers est toujours locale au serveur qui exécute l'application ; seule l'adresse publique (`PIECE_JOINTE_STORAGE_URL`) change d'un environnement à l'autre.

2 — Sauvegarder ce dossier avec la base : les pièces jointes ne sont pas dans la base de données. Voir la procédure de maintenance, §1.

Étape 6 — Envoi d'emails (SMTP)

Ce que fait cette étape : permettre à Passerelle d'envoyer des emails : invitation des nouveaux utilisateurs (candidats, chargés de mission) avec leur mot de passe provisoire, et lien de réinitialisation d'un mot de passe oublié.

Avant de commencer :

- ☐ Étape 4 terminée (panneau des variables connu).
- ☐ Un mot de passe choisi pour le compte email d'envoi (règle des mots de passe).

⚠ Sans SMTP, aucun email ne part. Les comptes sont créés, mais les invitations restent « en attente » (`pending_smtp_configuration`) : les nouveaux utilisateurs ne reçoivent jamais leur mot de passe.

1 — Créer un compte email d'envoi

(où : *cPanel*)

cPanel → **Comptes de messagerie** → **Créer** : une adresse d'expédition (ex. `noreply@<DOMAINE>`) et son mot de passe.

2 — Récupérer les réglages du serveur d'envoi

(où : *cPanel*)

En face du compte créé, cliquer sur **Configurer le client de messagerie** et relever : le **nom d'hôte SMTP**, le **port** et le type de **chiffrement**.

3 — Déclarer les 5 variables

(où : *cPanel* — *Setup Node.js App* → *Environment variables*)

Ajouter, puis **Enregistrer** :

Nom	Valeur
<code>SMTP_HOST</code>	Nom d'hôte SMTP relevé à l'instruction 2
<code>SMTP_PORT</code>	Port relevé à l'instruction 2
<code>SMTP_USER</code>	L'adresse email complète du compte d'envoi
<code>SMTP_PASSWORD</code>	Le mot de passe du compte d'envoi
<code>SMTP_FROM</code>	L'adresse d'expédition affichée aux destinataires (ex. <code>noreply@<DOMAINE></code>)

Le premier envoi réel se vérifie à l'étape 10 (réception d'un email d'invitation).

Étape 7 — Démarrage et publication de l'application

Ce que fait cette étape : démarrer le backend, puis compiler et publier l'interface, pour que `https://<DOMAINE>` affiche l'application.

Avant de commencer :

- ☐ Étapes 2 à 6 terminées (code compilé, base chargée, variables saisies).
- ☐ Une session SSH ouverte avec l'environnement Node activé (étape 2.3).

7.1 Démarrer le backend et le vérifier

1 — Redémarrer l'application

(où : *cPanel*)

cPanel → **Setup Node.js App** → l'application → bouton **Redémarrer** (« RESTART »).

💡 Sans redémarrage, l'ancien code continue de tourner même après une compilation.

2 — Vérifier que l'API répond

(où : *bash*, sur le serveur)

```
curl -s https://<DOMAINE>/api/healthz
```

Résultat attendu : `{"status": "ok"}`.

💡 `curl` envoie une requête web depuis le terminal, comme un navigateur, et affiche la réponse brute.

⚠ **Si la réponse n'est pas `{"status": "ok"}`**, reproduire le lancement à la main pour voir le vrai message d'erreur de Node.js, invisible autrement sur un hébergement mutualisé :

```
cd ~/passerelle-api
export PORT=3000
export DATABASE_URL='postgresql://...' # guillemets SIMPLES obligatoires
export JWT_SECRET='...'
node artifacts/api-server/dist/index.mjs
```

Le message affiché (`PORT` manquante, variable de score absente, connexion à la base refusée...) indique la cause. Arrêter avec `Ctrl+C`. La liste des erreurs fréquentes est en fin de document (Annexe).

7.2 Compiler l'interface

(où : *bash*, sur le serveur)

```
cd ~/passerelle-api/artifacts/onboardcc
export PORT=5000
export BASE_PATH=/
export VITE_API_BASE_URL=/api
pnpm run build
```

Résultat attendu : ✓ `built in ...` et des fichiers créés dans `dist/public/`. Des avertissements sur la taille des fichiers ou des « sourcemap » sont normaux.

💡 `BASE_PATH=` détermine comment les chemins des fichiers JavaScript et CSS sont écrits dans la page : une mauvaise valeur casserait le chargement de l'interface. `PORT` est nécessaire pour que le fichier de configuration s'évalue, sans effet sur le résultat.

7.3 Publier l'interface

(où : *bash*, sur le serveur)

```
cp -r ~/passerelle-api/artifacts/onboardcc/dist/public/* <DOSSIER_PUBLIC>/
```

💡 Le dossier public du domaine n'est **jamais** lu directement depuis le code : sans cette copie, une mise à jour du code ne change rien à ce que voient les visiteurs. Cette commande remplace les fichiers du même nom, ce qui est voulu.

7.4 Configurer Apache (première installation uniquement)

(où : *bash*, sur le serveur)

1 — Vérifier si la configuration est déjà en place

```
grep -c "cohabitation frontend statique" <DOSSIER_PUBLIC>/.htaccess
```

- Résultat **1** → déjà en place : **ne pas refaire l'instruction 2** (elle créerait un doublon).
- Résultat **0** (ou fichier introuvable) → faire l'instruction 2.

2 — Ajouter les règles à la fin du fichier **.htaccess**

```
cat >> <DOSSIER_PUBLIC>/.htaccess << 'EOF'

# --- Passerelle : cohabitation frontend statique + backend Node ---
RewriteEngine On
RewriteCond %{REQUEST_URI} ^/(api|uploads)/ [OR]
RewriteCond %{REQUEST_FILENAME} -f [OR]
RewriteCond %{REQUEST_FILENAME} -d
RewriteRule ^ - [L]
RewriteRule ^ /index.html [L]
EOF
```

💡 Ces règles ajoutent (sans jamais remplacer) un filet de sécurité au fichier déjà généré par Node.js
Selector : **/api**, **/uploads** ou un vrai fichier restent inchangés ; toute autre adresse est renvoyée vers **index.html**, indispensable pour les pages internes de l'interface (ex. **/login**).

7.5 Vérifier dans le navigateur

(où : *navigateur*)

1. Ouvrir **https://<DOMAINE>/** puis faire **Ctrl+F5** (rechargement sans cache).
2. Résultat attendu : l'**écran de connexion** de Passerelle s'affiche, sans erreur.

⚠ Un navigateur peut conserver l'ancienne version de l'interface en cache : sans **Ctrl+F5**, une mise à jour peut sembler ne rien avoir changé.

Étape 8 — Créer le premier compte administrateur

Ce que fait cette étape : créer le tout premier compte, qui permettra ensuite de créer tous les autres depuis l'interface. Aucun écran ne permet de le faire : on l'insère directement en base, avec un mot de passe **haché** (jamais en clair).

Avant de commencer :

- ☐ Étapes 3 et 7 terminées (base chargée, application accessible).
- ☐ Le mot de passe de l'administrateur choisi (règle des mots de passe, **12 caractères minimum**, avec une minuscule, une majuscule, un chiffre et l'un des trois caractères `_ - .`, rien d'autre).
- ☐ L'identité de l'administrateur (nom, prénom, adresse email).
- ☐ Une session SSH ouverte avec l'environnement Node activé.

⚠ Deux terminaux différents sont utilisés dans cette étape — ne pas les confondre. Repérer l'invite : `$` = bash (commandes), `=>` = psql (requêtes SQL terminées par `;`). Taper une requête SQL dans bash provoque `syntax error near unexpected token '('` ; un `cd` ou un `node` tapé dans psql échoue.

💡 Un **hash** est une version illisible et irréversible d'un mot de passe : c'est lui, et jamais le mot de passe, qui est stocké en base. À la connexion, l'application compare le mot de passe saisi à ce hash.

8.1 Générer le hash du mot de passe

(où : *bash*, sur le serveur)

```
cd ~/passerelle-api/artifacts/api-server
read -s -p "Mot de passe admin : " PW; export PW
node -e "require('bcryptjs').hash(process.env.PW, 12).then(h=>console.log(h))"
unset PW
```

Commande	Ce qu'elle fait
<code>cd .../api-server</code>	Se place dans le dossier du backend. Indispensable : ailleurs, <code>node</code> répond <code>Cannot find module 'bcryptjs'</code>
<code>read -s -p "..." PW; export PW</code>	Demande le mot de passe sans l'afficher et le garde en mémoire : il n'est écrit dans aucune commande, donc absent de l'historique du terminal
<code>node -e "require('bcryptjs')..."</code>	Calcule et affiche le hash
<code>unset PW</code>	Efface le mot de passe de la mémoire

Résultat attendu : une ligne commençant par `$2a$12$` ou `$2b$12$`. **La copier** : elle sert en 8.4.

8.2 Ouvrir la base

(où : *bash*, puis *psql*)

```
cd ~/passerelle-api
psql -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle
```

Saisir le mot de passe de la base. L'invite devient *<COMPTE>_passerelle=>* : **on est dans psql** ; les instructions 8.3 à 8.5 s'y tapent.

8.3 Créer le contact

(où : *psql*)

```
INSERT INTO contact (crm_key, role, genre, nom_contact, prenom_contact,
tel_contact, email_contact)
VALUES ('USER_ADMIN1', 'ADMIN', 'M', 'NOM_A_CHOISIR', 'PRENOM_A_CHOISIR',
'0600000000', 'email@exemple.fr')
RETURNING id_contact;
```

Résultat attendu : un tableau avec un nombre (*id_contact*). **Le noter.**

8.4 Créer l'utilisateur

(où : *psql*)

Remplacer *HASH_COPIE_EN_8_1* par le hash copié en 8.1 (entre les apostrophes, sans espace) et *ID_NOTE* par le nombre noté en 8.3 :

```
INSERT INTO user_ (login, password, id_contact, role_applicatif)
VALUES ('admin.passerelle', 'HASH_COPIE_EN_8_1', ID_NOTE, 'ADMIN');
```

Résultat attendu : *INSERT 0 1*.

💡 Le hash contient des \$: c'est normal, garder les apostrophes. *role_applicatif* doit valoir exactement *'ADMIN'* : c'est cette colonne, et non *contact.role*, qui donne les droits réels dans l'application.

8.5 Vérifier, puis quitter psql

(où : *psql*)

```
SELECT login, role_applicatif, active FROM user_;
```

La ligne *admin.passerelle* | *ADMIN* | *t* doit apparaître. Quitter avec :

\q

8.6 Tester la connexion

(où : *navigateur*)

Sur l'écran de connexion de <https://<DOMAINE>/>, se connecter avec **admin.passerelle** et le mot de passe choisi. Résultat attendu : accès à l'écran d'administration (</admin/referentiels>).

Étape 9 — Sécurisation

Ce que fait cette étape : vérifier que le serveur est prêt à recevoir de vrais utilisateurs. À parcourir **avant toute mise à disposition réelle**, sur chaque nouvel environnement.

Avant de commencer :

- ☐ Étape 8 terminée (connexion administrateur possible).

9.1 Certificat HTTPS

(où : *cPanel* et *navigateur*)

1 — Vérifier le certificat

Ouvrir <https://<DOMAINE>/> : le navigateur ne doit afficher **aucun avertissement de sécurité** (« Votre connexion n'est pas privée », **NET::ERR_CERT_AUTHORITY_INVALID**).

2 — Si le certificat est « auto-signé » : en générer un vrai

cPanel → **SSL/TLS Status** : si la ligne du domaine affiche **Self-signed**, la cause est là. Puis : cPanel → **Let's Encrypt SSL** → **Générer un nouveau certificat** → sélectionner le domaine → laisser les options par défaut (les deux domaines cochés, méthode HTTP-01, clé RSA 2048 bits) → **Générer** (et non « Simuler la génération »).
Apache redémarre seul : attendre quelques secondes avant de retester.

💡 Un nouveau domaine reçoit d'abord un certificat auto-signé temporaire. Le système de certificats automatique (**AutoSSL**) **ignore volontairement** tout domaine qui possède déjà un certificat non émis par lui, même auto-signé : sans intervention, le problème ne se corrige jamais seul. Ce piège concerne tout nouveau domaine sur ce type d'hébergement.

9.2 Liste de contrôle

(où : *selon le point* — *cPanel*, *bash* ou *navigateur*)

Accès réseau

- ☐ Seuls les ports nécessaires sont ouverts au public (80 et 443 pour le web, 22 pour SSH) ; aucun autre port de l'application ou de la base n'est exposé.
- ☐ SSH par **clé uniquement** (pas de mot de passe), avec liste d'adresses autorisées (étape 1).
- ☐ PostgreSQL n'est accessible **que depuis le serveur** (**localhost**), jamais depuis l'extérieur.

HTTPS et DNS

- ☐ Certificat valide et actif (9.1).
- ☐ Redirection automatique de HTTP vers HTTPS.
- ☐ L'application envoie déjà l'en-tête HSTS ; vérifier qu'aucune configuration du serveur ne l'annule.
- ☐ DNS : uniquement les enregistrements nécessaires ; supprimer tout enregistrement obsolète pointant vers un ancien hébergement.

Fichiers et permissions

- ☐ Le dossier `.git` du code n'est accessible depuis aucune adresse web.
- ☐ Aucun fichier contenant un secret (`.env`, clés, mots de passe) dans un dossier servi publiquement : les secrets passent uniquement par les variables d'environnement de cPanel.
- ☐ Le dossier `uploads/` n'est pas listable (`Options -Indexes`, étape 5).
- ☐ Aucune permission excessive (jamais de `chmod 777`).

Comptes et secrets

- ☐ `JWT_SECRET` propre à cette installation, non réutilisé.
- ☐ Règle des mots de passe respectée partout.
- ☐ Aucun compte de test conservé avec un mot de passe faible ou connu (voir l'étape 10).

Versions

- ☐ Les versions de PostgreSQL et de Node.js du serveur sont notées ; signaler explicitement une version ancienne ou non maintenue (ex. PostgreSQL 9.6) : elle fonctionne avec le fichier `_pg96` mais n'est plus supportée par son éditeur.

Étape 10 — Recette et mise à disposition

Ce que fait cette étape : vérifier de bout en bout que l'application fonctionne, puis nettoyer les données de test avant de donner accès aux vrais utilisateurs.

Avant de commencer :

- ☐ Étapes 1 à 9 terminées.
- ☐ Un ou plusieurs fichiers CSV de **test** (postes et candidats) ; chaque candidat de test doit avoir une adresse email **unique** (c'est son identifiant de connexion).

10.1 Vérifications à dérouler

(où : *navigateur*)

#	Fonction à vérifier	Résultat attendu
1	Connexion administrateur	Accès à l'administration
2	Création d'un compte Recruteur (Admin → Nouvel utilisateur → Créer et inviter)	Le compte est créé

#	Fonction à vérifier	Résultat attendu
3	Réception réelle de l'email d'invitation	L'email arrive dans la boîte du destinataire (valide le SMTP, étape 6)
4	Connexion avec le nouveau compte Recruteur	Accès à l'espace recruteur
5	Import d'un fichier de postes (CSV de test)	Rapport de vérification « OK » puis import réussi
6	Import d'un fichier de candidats (CSV de test)	Rapport « OK », import réussi, invitations envoyées
7	Connexion candidat : édition de la fiche de vœux	La fiche s'édite et se soumet
8	Génération automatique des opportunités et de leur score	Des opportunités apparaissent avec leur note
9	Parcours jusqu'à l'affectation (proposer au CM → approuver → mettre en lien → accord de principe → accord définitif → décision)	Le candidat passe « Affecté »

10.2 Supprimer les données de test

(où : *bash*, puis *psql*)

Le script fourni `documentation/reset_base_test.sql` supprime les postes, les candidats et les contacts (chargés de mission, chargés de zone, mission, partenaire) créés pendant les tests, et **conserve** les référentiels et les comptes Administrateur et Recruteur.

1 — Ouvrir la base

```
cd ~/passerelle-api
psql -h localhost -U <COMPTE>_passerelle_app -d <COMPTE>_passerelle
```

2 — Charger le script (dans *psql*)

```
\i documentation/reset_base_test.sql
```

3 — Lire l'aperçu affiché, puis valider ou annuler (dans *psql*)

Le script s'arrête avant de valider : lire les nombres affichés, puis taper soit `COMMIT` ; pour appliquer, soit `ROLLBACK` ; si le résultat surprend.

4 — Quitter *psql*

```
\q
```

⚠ Ne pas exécuter ce script une fois de **vrais** candidats importés : il les supprimerait.

Annexe — Erreurs fréquentes

Message ou symptôme	Cause	Que faire
<code>Connection timed out</code> à la connexion SSH	Adresse IP du poste non autorisée	1.4 : ajouter l'adresse IP actuelle
<code>-bash: pnpm: command not found</code>	Environnement Node non activé dans cette session	Refaire la commande <code>source .../activate</code> (2.3)
<code>fatal: not a git repository</code>	<code>git pull</code> lancé hors du dossier du code	<code>cd ~/passerelle-api</code>
<code>destination path ... already exists and is not an empty directory</code>	<code>git clone</code> sur un dossier non vide	Cas B (2.2) : <code>git pull</code> au lieu de <code>git clone</code>
<code>Cannot find module 'bcryptjs'</code>	<code>node</code> lancé hors du dossier du backend	<code>cd ~/passerelle-api/artifacts/api-server</code> (8.1)
<code>syntax error near unexpected token '('</code>	Requête SQL tapée dans bash	Ouvrir psql d'abord (invite =>)
la relation « ... » n'existe pas juste après le chargement	Session psql restée après <code>\i</code> (chemin de recherche vide)	Préfixer par <code>public.</code> (3.6) ou quitter et se reconnecter
Des centaines de « existe déjà » au chargement du fichier	Fichier chargé deux fois	Nettoyer (3.4) puis un seul chargement
<code>role "postgres" does not exist / syntax error</code> au chargement	Mauvaise version du fichier pour le serveur	Utiliser la version <code>_pg96</code> (3.5)
authentification par mot de passe échouée (psql)	Mot de passe mal saisi (rien ne s'affiche)	Recoller le mot de passe sans espace ; vérifier celui de cPanel et de <code>DATABASE_URL</code>
<code>PORT environment variable is required / « Could not spawn process »</code>	Variable <code>PORT</code> manquante	4.3 : ajouter <code>PORT=3000</code>
<code>Invalid URL</code> au démarrage	Caractère spécial dans le mot de passe de <code>DATABASE_URL</code>	Choisir un mot de passe sans caractère spécial
Configuration de <code>scoring</code> absente ou invalide : X	Variable <code>MATCH_SCORE_*</code> manquante	4.4 : comparer avec <code>.env.example</code>
Page blanche ou ancienne interface	Cache du navigateur, copie oubliée ou	<code>Ctrl+F5</code> ; refaire 7.2 et 7.3

Message ou symptôme	Cause	Que faire
	mauvais <code>BASE_PATH</code>	
Emails d'invitation qui n'arrivent pas	SMTP non configuré	Étape 6
<code>NET::ERR_CERT_AUTHORITY_INVALID</code>	Certificat auto-signé	9.1

Fin de l'installation

L'application est installée. Pour tout geste à partir de maintenant — sauvegarder, déployer une mise à jour, diagnostiquer un signalement, changer un mot de passe — voir la procédure de maintenance : [2-procedure_maintenance_passerelle.md](#).

Historique des versions

Version	Date	Modification
1.x	07/09/2026 – 25/09/2026	Rédaction et améliorations successives à partir des premières installations.
2.0	25/09/2026	Refonte complète en livrable : introduction (objectif, grandes étapes, conventions), une étape par grande phase (« Avant de commencer », instructions numérotées, terminal précisé pour chaque commande), cas « première installation » et « réinstallation », erreurs fréquentes en annexe.